

Personal Expense Tracker

The **Personal Expense Tracker** is a lightweight, web-based software application designed to help individuals track daily spending, set monthly category budgets, analyze trends, and export financial data.

Built on Python, Flask, and SQLite, the application balances simplicity with modern interactive design featuring a dynamic dashboard, visual charts, custom Indian Rupee'₹' formatting, and contextual emoji-labeled subcategories.

Problem Statement:

Personal spending is frequently recorded inconsistently or ignored completely. Unstructured notes or loose receipts fail to answer critical personal finance questions regarding spending by category, trends over time, or adherence to monthly budgets.

Primary Objectives:

- **Effortless Expense Management:** Streamline adding, editing, deleting, filtering, and searching expense entries.
- **Budget Tracking:** Enable users to define and monitor monthly spending caps across predetermined categories.
- **Visual Insights:** Provide real-time dashboards displaying summary cards, averages, daily trends, and category breakdowns.
- **Account Isolation:** Ensure secure multi-user isolation so each user's financial records remain private.
- **Data Portability:** Allow users to export filtered records into standardized CSV files.

Application Architecture & Tech Stack:

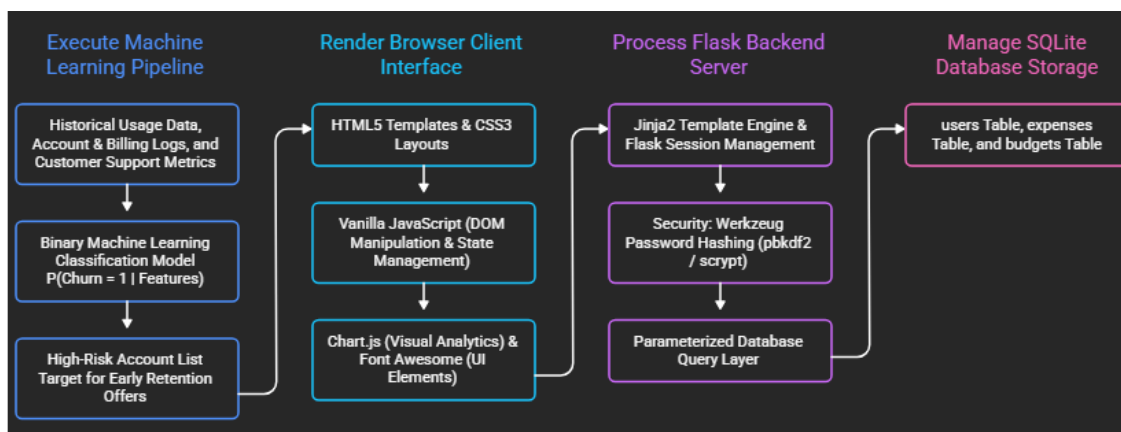


Fig1: Application Architecture

Technology Breakdown:

Component	Technology	Purpose
Backend Framework	Python / Flask	Web routing, session handling, API endpoints, and template rendering
Database	SQLite (sqlite3)	Persistent relational storage for user accounts, transactions, and budgets
Security	Werkzeug Utilities	Secure password hashing (generate_password_hash, check_password_hash)
Frontend UI	HTML5, CSS3, JS	Interactive client-side interface and asynchronous API integration
Data Visualization	Chart.js	Dynamic client-side rendering of charts and breakdown graphs
UI Components	Font Awesome / Google Fonts	CDN-loaded typography and icon sets

Database Schema & Data Model:

The application utilizes three core entities. Metadata for categories (names, icons, colors, and emoji subcategory options) is configured in application code for performance and consistency.

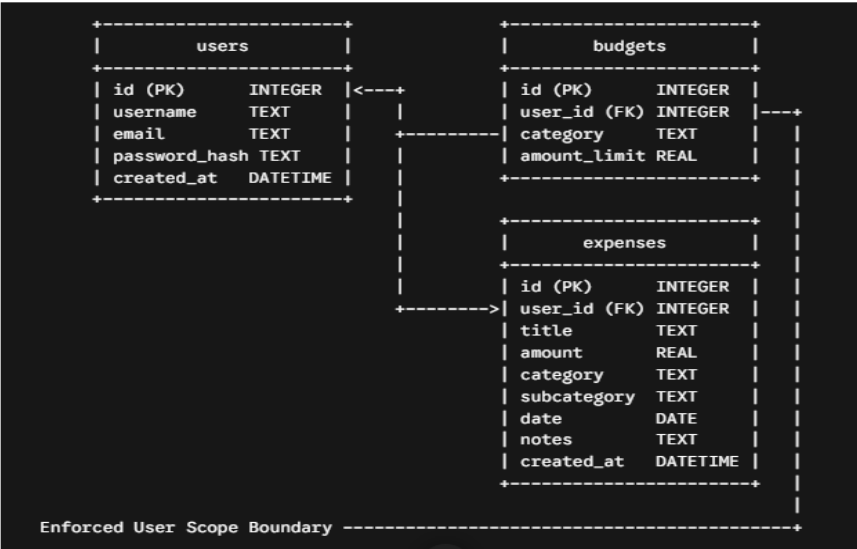


Fig2: Data Model

1. users Table

- `id` (INTEGER, Primary Key, Auto-increment) – Unique user identifier.
- `username` (TEXT, Unique, Not Null) – Unique handle chosen during registration.
- `email` (TEXT, Unique, Not Null) – User's email address
- `password_hash` (TEXT, Not Null) – Securely hashed user password.
- `created_at` (DATETIME, Default CURRENT_TIMESTAMP) – Registration timestamp.

2. expenses Table

- `id` (INTEGER, Primary Key, Auto-increment) – Transaction record ID.
- `user_id` (INTEGER, Foreign Key > `users.id`) – Scopes record to specific user.
- `title` (TEXT, Not Null) – Brief title describing the transaction.
- `amount` (REAL, Not Null) – Numerical expense amount in Indian Rupees (₹).
- `category` (TEXT, Not Null) – Main expense category.
- `subcategory` (TEXT, Optional) – Emoji-labeled subcategory selection.
- `date` (DATE, Not Null) – Transaction date.
- `notes` (TEXT, Optional) – Additional transaction details or description.
- `created_at` (DATETIME, Default CURRENT_TIMESTAMP) – Entry timestamp.

3. budgets Table

- `id` (INTEGER, Primary Key, Auto-increment) – Budget entry ID.
- `user_id` (INTEGER, Foreign Key > `users.id`) – Scopes budget to specific user.
- `category` (TEXT, Not Null) – Associated category name.
- `amount_limit` (REAL, Not Null) – Assigned monthly budget threshold (₹).

System Features & Dynamic Data Flow:

Key Features:

1. **Account Authentication:** Registration, password-hashed login, and session-based logout.
2. **Transaction Management:** Full CRUD (Create, Read, Update, Delete) capability for individual transactions.
3. **Category-Aware Emoji Suggestions:** Selecting a category populates context-specific subcategory options with emojis.
4. **Analytics Dashboard:** Displays spending totals, daily averages, trend line charts, and category distribution pie/doughnut charts.
5. **Budget Monitoring:** Progress indicators comparing monthly totals against set category limits.
6. **Data Search & Filter:** Filter transactions by date range, category, subcategory, or keyword search.
7. **CSV Exporting:** Download expense records matching current filters directly into CSV format.

End-to-End User & Data Flow:

1. **Authentication:** User logs in. Flask verifies credentials using `check_password_hash` and initializes a signed session cookie.
2. **Dashboard Initialization:** Client loads JavaScript, requesting user metadata, categories, expenses, and analytics via API endpoints.
3. **Database Querying:** Flask extracts `user_id` from the active session and executes scoped parameterized SQL queries (e.g., `WHERE user_id = ?`).
4. **Client-Side Rendering:** Received JSON populates summary cards, updates Chart.js figures, formats amounts as '₹', and populates transaction tables.
5. **Expense Creation:**
 - User submits new transaction details.
 - Client posts data to API.
 - Flask validates inputs and saves to database with active `user_id`.
 - UI refreshes data tables, budgets, and visual charts automatically.

Data Control & Security Measures:

- **Password Security:** Store salted/hashed passwords using Werkzeug hashing algorithms rather than plaintext.
- **SQL Injection Prevention:** Pass SQL parameters as query arguments (?) rather than concatenated strings.
- **User Data Scoping:** Every database query affecting expenses or budgets enforces strict `WHERE user_id = ?` filters.
- **Cookie Protection:** Session cookies are configured with `HttpOnly` and `SameSite=Lax` flags to minimize cross-site scripting (XSS) and cross-site request forgery (CSRF) risks.
- **Input Validation:** Backend validation checks required fields, email shapes, username constraints, and positive numeric inputs for spending and budget limits.

Findings & Project Evaluation:

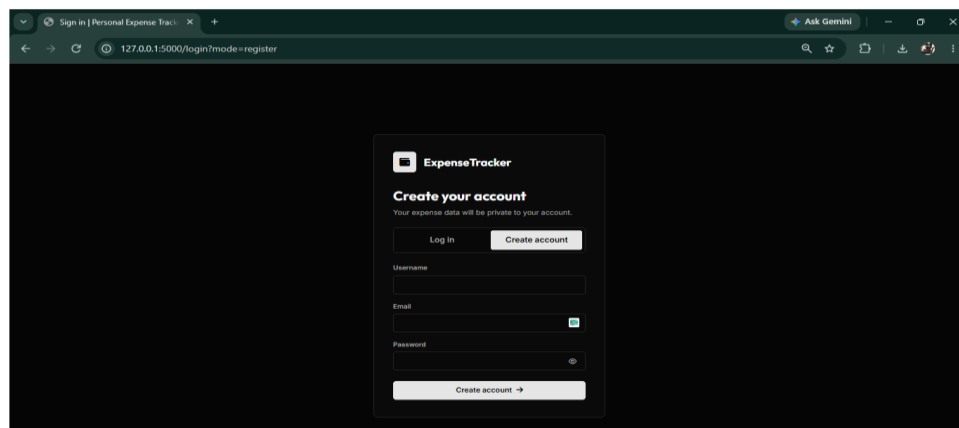


Fig3: Create Account

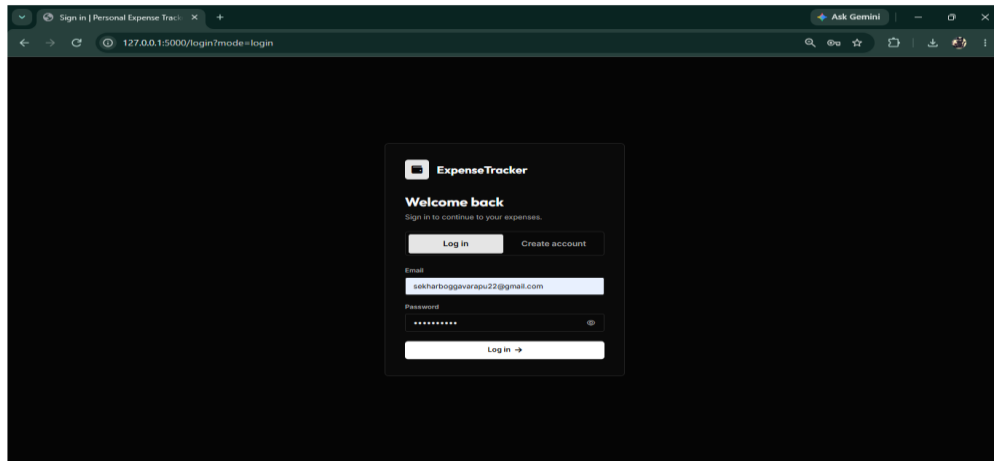


Fig4: Login Page

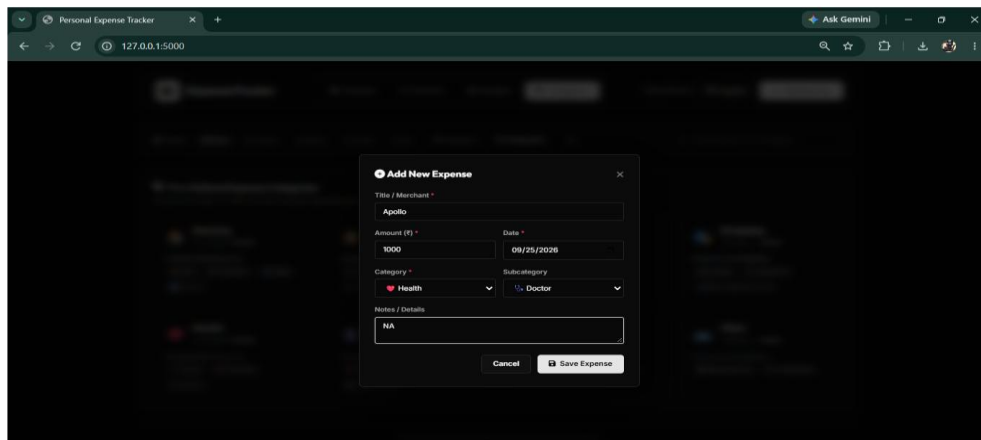


Fig5: Add Expense

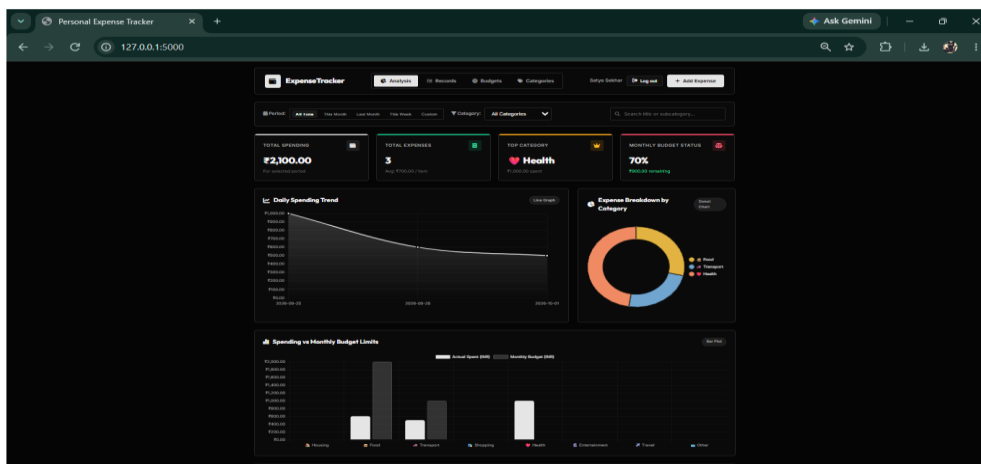


Fig6: Analysis Dashboard

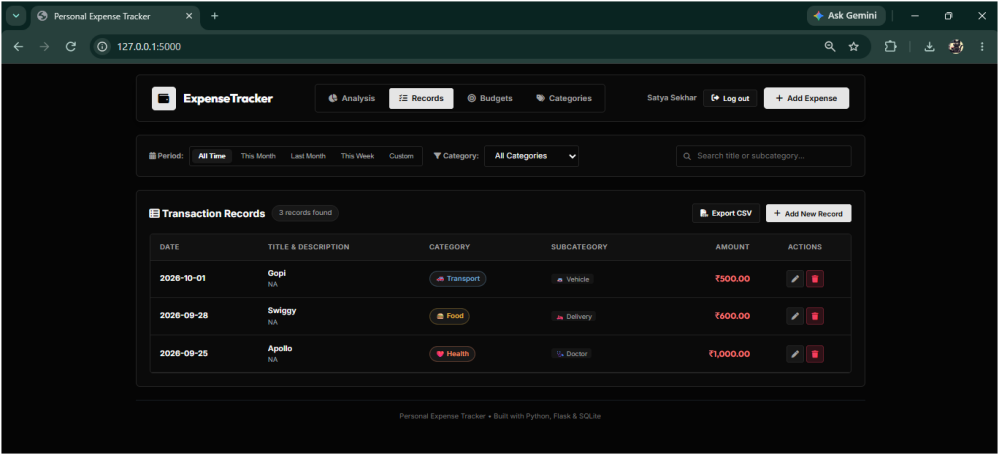


Fig7: Records Dashboard

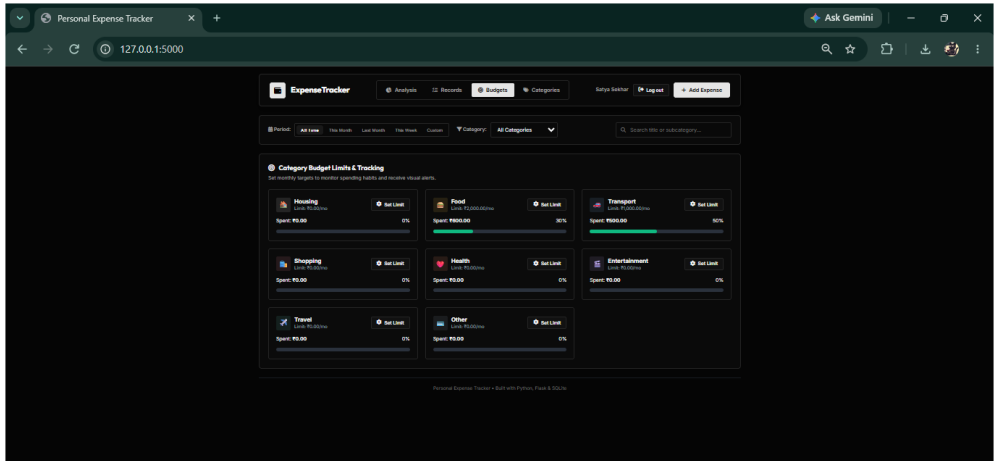


Fig8: Budget Dashboard

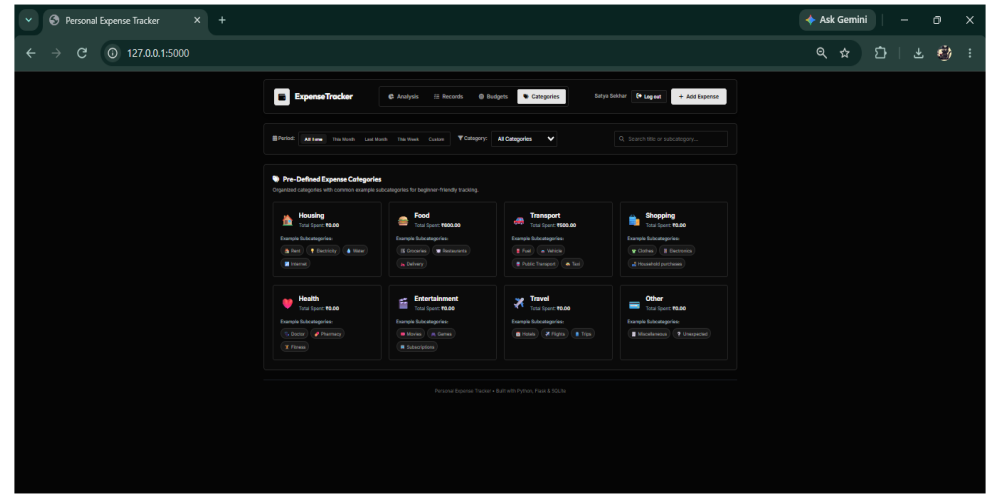


Fig9: Categories Dashboad

	title	amount	category	subcateg...	date	notes	created_at	user_id
1	Gopi	500	transport	Vehicle	2026-10-01	NA	2026-09-30 13:...	
2	Swiggy	600	food	Delivery	2026-09-28	NA	2026-09-30 13:...	
3	Apollo	1000	health	Doctor	2026-09-25	NA	2026-09-30 14:...	

Fig10: ExpenseDB

Strengths

- **Zero External DB Dependencies:** SQLite integration allows easy local setup without additional database server overhead.
- **User Isolation:** Secure session management ensures individual financial data remains separated.
- **Rich UI Features:** Integrated visualization, search/filter, dynamic subcategory loading, and localized currency formatting (₹) provide a polished user experience.

Technical Limitations

- **CDN Reliance:** External dependencies (Chart.js, Font Awesome, Google Fonts) require an active internet connection unless locally bundled.
- **Static Categories:** Spending categories are predefined in backend code rather than customized per user in the database.
- **Deployment Constraints:** Uses default session secret management unless configured via `SECRET_KEY` environment variable

Future Enhancements

1. **Enhanced Authentication & Security:** Implement email verification, self-service password resets, explicit CSRF tokens, and account deletion options.
2. **Custom Categories & Recurrence:** Allow users to create custom categories and set up automatic recurring expenses (e.g., monthly subscriptions, rent).
3. **Advanced Reporting & Imports:** Support importing transactions via bank CSV files and generating downloadable PDF monthly reports.
4. **Production Readiness:** Transition to HTTPS deployment, utilize secure secret key management, implement multi-currency conversion features, and set up automated database backup procedures.